

- 现在我已经有了足够的信息来提供全面的函数级通信调用链分析。以下是完整图景。

适配仓(dsar-hq) 与 平台仓(dsar-hq-plat) 通信函数调用关系全景

一、整体分层架构



二、SOC侧（dsar_app）通信调用链

2.1 车辆代理主循环 → 平台信号路由

文件: proxy_soc.cpp

```
oem_proxy_init() // 适配仓：代理初始化入口
├─ proxy_init() // 初始化radar/uss proxy
└─ add_period_work_with_node_name( // 平台仓函数 [dsar-plat-bf: init.h]
    "com_proxy_thread", ...,
    proxy_main_function, ...) // 注册100Hz周期任务

proxy_main_function() // 适配仓：100Hz主循环
└─ radar_proxy_main_function() // 适配仓：20Hz雷达代理处理
```

2.2 自动生成代码 → 平台信号路由引擎

生成目录: proxy/x_dom_can_rt_gen_com/ (com服务路由)

生成目录: proxy/x_dom_can_rt_gen_uds/ (uds服务路由)

```
// 适配仓生成代码调用平台仓的sig_route
SigIf_Get_VehicleSpeed_oABS_1_0xc0_odefault(float *signal_phy)
├─ 读取全局 sig_route_read_tx_rx_sigs_dji_application 结构体
└─ 该结构体由平台仓 sig_route_main_function() 周期性更新

SigIf_Set_HAD_SteeringWheelAngleRequest_oHAD_3_0x11a_odefault(float *signal_phy)
├─ 写入全局 sig_route_write_to_mcu_sigs_xxx 结构体
└─ 该结构体由平台仓 sig_route_main_function() 周期性发送到MCU
```

2.3 平台仓核心：跨域CAN路由引擎

文件: dsar-plat-bf/dsar_app/cdd/x_dom_can_rt/sig_route_soc.cpp

文件: dsar-plat-bf/dsar_app/cdd/x_dom_can_rt/msg_route_soc.cpp

文件: dsar-plat-bf/dsar_app/cdd/x_dom_can_rt/x_dom_can_rt_soc.cpp

```
// 平台仓：每个SOC进程启动时调用
x_dom_can_rt_init("com") // com_app进程初始化
x_dom_can_rt_init("ad") // ad_app进程初始化
x_dom_can_rt_init("doipv4") // uds_v4进程初始化
├─ dcms_adp_topic_register() // 平台仓：注册DCMS Topic
├─ // 注册/x_dom_can_rt/mcu_tx_rx_msgs (MCU-SOC消息)
├─ // 注册/x_dom_can_rt/write_to_mcu_msgs (SOC-MCU消息)
└─ // 注册/x_dom_can_rt/write_sigs (跨进程信号写入)

// 平台仓：周期性主函数（每50ms由DCOS调度触发）
x_dom_can_rt_main_func()
├─ msg_route_main_function() // 平台仓：CAN消息路由
├─ └─ 从DCMS Topic接收MCU发来的CAN消息
├─ └─ 超时检测：标记超时消息状态为 X_DOM_CAN_RT_TIMEOUT
├─ └─ 更新 CanMsg 对象池（每帧CAN数据+状态）
├─
├─ sig_route_main_function() // 平台仓：信号级路由
├─ └─ sig_route_send_mcu_tx_rx_sigs_proc()
├─ └─ └─ sig_route_send_mcu_tx_rx_sigs()
├─ └─ └─ dcms_mcu_topic_send_msg() // 平台仓：通过DCMS转发MCU信号
├─ └─ └─ // 将mcu_tx_rx_sigs发送给ad_app/uds_app进程
├─
├─ sig_route_send_write_to_mcu_sigs_proc()
├─ └─ sig_route_send_write_to_mcu_sigs()
├─ └─ └─ dcms_mcu_topic_send_msg() // 平台仓：将SOC信号下发MCU
├─ └─ └─ // 通过/x_dom_can_rt/write_to_mcu_msgs Topic发送
```

2.4 平台仓DCMS Topic注册表

文件: dsar-plat-bf/dsar_app/cdd/x_dom_can_rt/x_dom_can_rt_dcms_config_com_app.h

定义了三个进程各自的DCMS Topic配置宏:

- `DCMS_X_DOM_CAN_RT_AD_APP_TOPIC_CONFIG` — AD应用进程的跨域CAN Topic
- `DCMS_X_DOM_CAN_RT_COM_APP_TOPIC_CONFIG` — COM服务进程的跨域CAN Topic
- `DCMS_X_DOM_CAN_RT_UDS_APP_TOPIC_CONFIG` — UDS诊断进程的跨域CAN Topic

内部Topic命名 (定义在 `x_dom_can_rt_internal.h`):

```
/x_dom_can_rt/mcu_tx_rx_msgs      → MCU-SOC: MCU收到的CAN消息上报
/x_dom_can_rt/write_to_mcu_msgs   → SOC-MCU: SOC要写入MCU的CAN消息
/x_dom_can_rt/write_to_vip_msgs   → SOC-VIP: com转发给VIP的CAN消息
/x_dom_can_rt/write_sigs          → 跨进程: 其他进程写入信号给com服务
/x_dom_can_rt/read_sigs/mcu_to_ad_app → P2P: MCU直接发给AD_APP
/x_dom_can_rt/write_sigs/ad_app_to_mcu → P2P: AD_APP直接发给MCU
```

2.5 SOC侧UDS诊断通信栈

文件: `entry/CMakeLists.txt`

```
// doip_service可执行文件, 各自独立链接
dji_doip_service_ipv4 → 链接 dsar_plat_config + mini_dcos + xml2
dji_doip_service_ipv6 → 链接 dsar_plat_config + mini_dcos + xml2
└─ 初始化时调用 x_dom_can_rt_init("doipv4"/"doipv6")
└─ 周期性调用 x_dom_can_rt_main_func()
└─ 通过DCMS Topic与COM服务交换诊断CAN消息
```

三、MCU侧 (dsar_fw) 通信调用链

3.1 车辆主任务调度 → 各域处理

文件: `xwire/vehicle.c`

```
vehicle_init() // MCU初始化
└─ vehicle_lat_init() // 横向控制设备注册
└─ lat_dev_register(&vehicle_lat) // 平台仓 [dsar-bsw]: 设备注册
└─ vehicle_lon_init() // 纵向控制
└─ vehicle_time_init() // 时间同步
└─ vehicle_planning_init() // 规划数据
└─ vehicle_perception_init() // 感知数据
└─ vehicle_sensor_init() // 传感器数据
└─ vehicle_app_init() // 应用数据
└─ app_dev_register(&vehicle_app) // 平台仓 [dsar-bsw]
└─ zyt_vehicle_info_init() // 车辆信息
└─ zyt_oemdata_ctrl_msg_init() // OEM控制消息
```

```
vehicle_main_function() // 100Hz主循环
└─ vehicle_lat_main_function(tick) // 横向控制100Hz
└─ vehicle_lon_main_function(tick) // 纵向控制
└─ vehicle_time_main_function(tick) // 时间同步5Hz
└─ vehicle_planning_main_function(tick) // 规划数据处理
└─ vehicle_perception_main_function(tick) // 感知数据
└─ vehicle_sensor_main_function(tick) // 传感器数据
└─ vehicle_app_main_function(tick) // 应用数据处理50Hz
└─ vehicle_info_main_function(tick) // 车辆信息发布
└─ oemdata_ctrl_msg_main_function(tick) // OEM控制消息处理
```

3.2 横向控制通信: CAN信号读取 → 控制指令下发

文件: `xwire/vehicle_lat.c`

```
vehicle_lat_main_function(tick) // 100Hz
└─ vehicle_lat_get() // 读取CAN信号 (每周期)
    // ===== AUTOSAR Com Stack 读取 =====
    Com_ReceiveSignal(ComConf_ComSignal_EPS1_SteeringWheelAngle_xxx_Rx, &var)
    Com_ReceiveSignal(ComConf_ComSignal_VehicleSpeed_xxx_Rx, &var)
    // ... 20+ 个CAN信号读取
    // 这些 ComConf_ComSignal_xxx_Rx 符号来自 dsar-bsw 子模块
    // 定义在 microsar_config/Appl/GenData/Com_Cfg.h
    └─ ctrlget_lati_acc_fdbk() // 加速度物理值转换
    └─ ctrlget_lati_yawrate_fdbk() // 横摆角速度转换
    └─ ctrlget_steer_angle_fdbk() // 方向盘角度转换(含符号/有效位)
    └─ ctrlget_steer_angle_speed_fdbk() // 方向盘角速度转换
    └─ ctrlget_steer_hand_torque_fdbk() // 手力矩转换

└─ vehicle_lat_set() // 下发控制指令 (每周期)
    └─ vehicle_lat_driving_cmd() // 行车模式横向指令
        └─ 根据EPS状态机填充 HAD_ControlReqSt_EPS
        └─ HAD_SteeringWheelAngleRequest 等TX变量

    └─ vehicle_lat_parking_cmd() // 泊车模式横向指令

    // ===== 生成代码: 写入MCU信号 =====
    SigIf_Set_HAD_ControlReqSt_EPS_oHAD_2_0x119_odefault(&val)
    SigIf_Set_HAD_SteeringWheelAngleRequest_oHAD_3_0x11a_odefault(&val)
    SigIf_Set_HAD_SteeringWheelAngleSignRequest_oHAD_3_0x11a_odefault(&val)
    SigIf_Set_HAD_SteeringWheelBlendingGain_oHAD_3_0x11a_odefault(&val)
    // → 这些SigIf_Set函数写入全局msg buffer
    // MCU侧x_dom_can_rt引擎周期性通过X-Wire发送给SOC
```

3.3 AUTOSAR Com Stack 信号定义宏

文件: `microsar_config/Appl/GenData/Com_Cfg.h`

```
// dsar-bsw子模块生成的AUTOSAR Com信号句柄 (示例)
#define ComConf_ComSignal_EPS1_SteeringWheelAngle_oEPS1_SAS_1_oHH_HA1PPVCar_ADAS_Backbone_CANFD_B0faf4f1_Rx ...
#define ComConf_ComSignal_VehicleSpeed_oABS_1_oHH_HA1PPVCar_ADAS_Backbone_CANFD_XXX_Rx ...

// 命名规则: ComConf_ComSignal_{信号名}_o{报文名}_o{网络名}_{hash}_Rx/Tx
```

函数 `Com_ReceiveSignal()` 和 `Com_SendSignal()` 来自 `dsar-bsw` 子模块的AUTOSAR COM栈实现, 底层通过CanIf -> PduR -> Can驱动程序实际收发CAN帧。

3.4 MCU侧自动生成代码: 跨域路由配置

目录: `x_dom_can_rt_config/`

```
// ===== 配置文件 x_dom_can_rt_cfg.h =====
// 外挂CAN消息buffer (MCU要发给VIP/SOC的完整CAN帧)
extern uint8_t mcu_write_to_vip_msg_ADV_2_default_0x0x100_buf[32];
extern uint8_t mcu_write_to_vip_msg_HAD_2_default_0x0x119_buf[16];
extern uint8_t mcu_write_to_vip_msg_HAD_3_default_0x0x11a_buf[32];
// ... 9条write_to_vip消息buffer

// 消息路由表
extern can_msg_base_with_cycle_t msg_route_mcu_write_to_vip_msg_tbl[9]; // VIP消息表
extern can_msg_filter_item_with_e2e_t msg_route_rx_msg_filters[30]; // RX过滤器
extern const can_msg_filter_item_t msg_route_tx_msg_filters[4]; // TX过滤器
extern const can_sig_base_info_mcu_with_e2e_t msg_route_write_sig_tbl[34]; // 信号写入表

// 初始化函数
int32_t msg_route_mcu_write_to_vip_msg_init(void);

// ===== 信号接口 x_dom_can_rt_sigif.h/c =====
// 从SOC侧写入MCU的SigIf_Set函数 (write_to_vip方向)
SigIf_Set_HAD_ControlReqSt_EPS_oHAD_2_0x119_odefault(uint8_t *signal_phy)
└─ 写入 mcu_write_to_vip_msg_HAD_2_default_0x0x119_buf
└─ 平台仓 [dsar-bsw] 的 Com_SendSignal() 将buffer发送到CAN总线

// MCU读取CAN信号上报SOC的SigIf_Get函数 (read_mcu方向)
SigIf_Get_VehicleSpeed_oABS_1_oADAS(float32_t *signal_phy)
└─ 从MCU全局信号池读取物理值
```

3.5 MCU SOME/IP -> CAN 桥接

文件: `com_if/someip_if.c`

```
someip_if_init() // app_init_stage2阶段自动调用
└─ someip_to_can_mrr_init()
    └─ dcms_mcu_topic_setup_callback( // 平台仓 [dsar-plat-bf]
        DCMS_TOPIC_SOMEIP_TO_CAN_MRR_RX,
        someip_to_can_mrr_rcv_cb, ...) // 注册SOMEIP-CAN数据接收回调

someip_to_can_mrr_rcv_cb(data, len) // SOC侧SOMEIP数据到达回调
└─ memcpy(&stoc_mrr_mcu, data, ...) // 拷贝雷达SOMEIP-CAN结构体
└─ 结构体内容后续通过CAN发送到车辆总线

faw_someip_dcms_send(topic_id, ptr, len) // MCU-SOC SOMEIP数据发送
└─ sync_signal_aligned_time_us_by_freq() // 平台仓: 时间对齐
└─ somip_data_dcms_mcu_send()
    └─ dcms_mcu_topic_send_msg() // 平台仓: DCMS发送
```

3.6 MCU侧代理传感器数据上报

文件: `proxy_comm/proxy_sensor/mradar_obtain.c`

文件: `proxy_comm/proxy_sensor/mradar_info_send.c`

```
mradar_obtain_main() // 从CAN总线获取毫米波雷达数据
└─ Com_ReceiveSignal() // 平台仓 [dsar-bsw]: AUTOSAR COM读取

mradar_info_send() // 通过DCMS发送给SOC
└─ dcms_mcu_topic_send_msg() // 平台仓 [dsar-plat-bf]
```

3.7 MCU DCMS Topic注册

文件: `adapter/dcms_adapter.c`

```
#include DCMS_CONFIG_PATH // 引入各项目的topic配置

static mcu_topic_service_info_t dcms_mcu_app_topics[] = DCMS_MCU_TOPIC_CONFIG;

DCMS_MCU_REG_TOPICS(dsar_dcms, dcms_mcu_app_topics);
// 宏展开为: 调用 dcms_mcu_topic_register("dsar_dcms", dcms_mcu_app_topics, count)
```

3.8 MCU侧信号传输匹配引擎

文件: `proxy_comm/comm/comm_sdt.c`

```
match_table_update(sensor_signal_trans, trans_table_size, signal_id)
```



```
match_table_update(sensor_signal_trans, trans_table_size, signal_id)
// 当某个CAN信号更新时，更新传输匹配表的AND/OR位掩码

sensor_signal_transmit_handler(sensor_signal_trans, trans_table_size)
// 检查AND/OR匹配条件是否满足，决定是否触发数据发送
// 用于传感器信号的按条件转发

3.9 MCU CAN Gateway适配

文件: adapter/cangateway_adapter.c

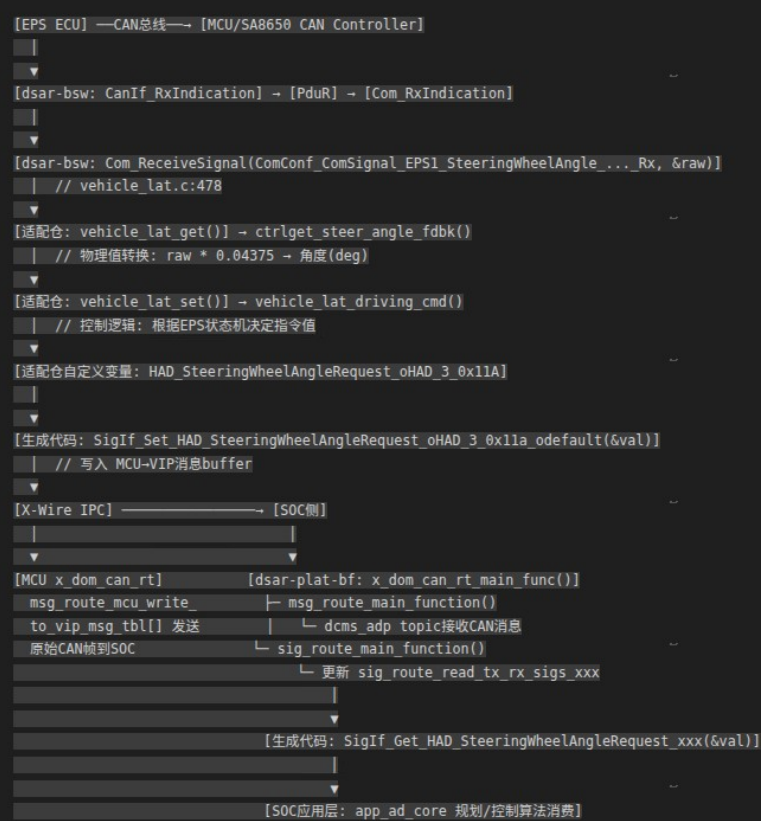
static can_router_tab_t cangateway_tx_id_cfgs[] = CAN_GATEWAY_TX_ID_ROUTER;

CAN_GATEWAY_REG_TX_ID_PARAMS(cangateway, cangateway_tx_id_cfgs, COMM_UDP_ETHNET_CAN_RECV_ID);
// 平台仓 [dsar-bsw]: 注册CAN网关路由表
// 功能: SOC-MCU方向的CAN帧转发，通过以太网UDP接收-CAN发送
```

四、平台仓(dsar-hq-plat) 关键函数分类总结

平台包	头文件	核心函数	作用
dsar-plat-bf	cdd/sig_route_soc.h	sig_route_init() / sig_route_main_function()	SOC侧信号级路由引擎
dsar-plat-bf	cdd/msg_route_soc.h	msg_route_init() / msg_route_main_function()	SOC侧CAN消息级路由
dsar-plat-bf	cdd/x_dom_can_rt_soc.h	x_dom_can_rt_init() / x_dom_can_rt_reg_msg_cb()	跨域CAN运行时SOC侧入口
dsar-plat-bf	cdd/x_dom_can_rt_com.h	数据结构定义 (can_msg_t, can_sig_base_info_mcu_t等)	跨域CAN数据序列化格式
dsar-plat-bf	cdd/dcms_mcu_api.h	dcms_adp_topic_register(), dcms_mcu_topic_send_msg(), dcms_mcu_topic_setup_callback()	DCMS Topic发布/订阅/回调
dsar-plat-bf	cdd/sig_handle_soc.hpp	SigHdl 模板类, signal_if_get(), signal_if_set()	CAN信号位操作(大小端转换)
dsar-plat-bf	cdd/init.h	add_period_work_with_node_name()	周期性任务注册
dsar-plat-bf	cdd/recorder.h	recorder_data()	数据录制
dsar-bsw	AUTOSAR_Com.h	Com_ReceiveSignal(), Com_SendSignal()	AUTOSAR COM栈信号读写
dsar-sip	x_dom_can_rt 扩展	SigIf_Get_xxx() / SigIf_Set_xxx()	跨域CAN信号接口自动生成

五、完整数据流向示例：EPS方向盘角度信号



<> vehicle_sensor.c

继续

Thought for 2s >

Read vehicle_sensor.c

Bash List directories in fawhq_e001_10 MCU product

IN find /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/fawhq_e001_10/ -type d | sort

OUT /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/fawhq_e001_10/
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/fawhq_e001_10/adapter
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/fawhq_e001_10/com_if

- **Bash** List all faw vehicle product variants

```
IN find /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/ -maxdepth 1 -type d | sort

OUT /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/fawhq_e001_10
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/fawhq_p301
```

- Thought for 5s >

- **Read** vehicle_sensor.c

- **Bash** List MCU proxy group files for e001_10

```
IN find /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/fawhq_e001_10/proxy/ -type f | sort

OUT /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/fawhq_e001_10/proxy/ActuatorGroup/ActuatorProxy.c
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/fawhq_e001_10/proxy/ActuatorGroup/ActuatorProxy.h
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/fawhq_e001_10/proxy/ActuatorGroup/BrakeUnitProxy_Types.h
```

- **Read** mradar_obtain.c

- **Bash** List ChassisGroup files for e001_10

```
IN find /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/fawhq_e001_10/proxy/ChassisGroup/ -type f | sort

OUT /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/fawhq_e001_10/proxy/ChassisGroup/ChassisProxy.c
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/fawhq_e001_10/proxy/ChassisGroup/ChassisProxy.h
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/fawhq_e001_10/proxy/ChassisGroup/ChassisProxy_Types.h
```

- Thought for 4s >

- **Read** proxy.c

- **Read** ChassisProxy.c

- **Bash** List xwire files for e001_10

```
IN find /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/fawhq_e001_10/xwire/ -type f | sort

OUT /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/fawhq_e001_10/xwire/Makefile
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/fawhq_e001_10/xwire/vehicle_app.c
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/fawhq_e001_10/xwire/vehicle_app.h
```

- Thought for 4s >

- **Bash** List directories for e001_10 SOC product

```
IN find /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/fawhq_e001_10/ -type d | sort

OUT /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/fawhq_e001_10/
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/fawhq_e001_10/app_adapt
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/fawhq_e001_10/autosar_adapter
```

- **Read** mradar_info_send.c

- **Read** vehicle_info.c

- Thought for 4s >

- 现在补充 MCU 侧传感器数据流、代理组模式、设备抽象层以及 SOC→MCU 方向的反向数据流。

六、MCU 侧代理组(Proxy Group)架构

e001_10 的 MCU 代理层按车辆域划分为 6 个代理组，位于 proxy/ 下，由 proxy.c:39-65 统一调度：

```
proxy_main_function() // 100Hz入口
├─ ActuatorProxy_Main() // 执行器组：20Hz — EPS/制动/驱动指令
├─ BCMProxy_Main() // 车身控制组：20Hz — 灯光/门窗/雨刮
├─ ChassisProxy_Main() // 底盘组：20Hz — ESC/EPB/悬架信号
├─ HMIProxy_Main() // 人机交互组：20Hz — 仪表/中控显示信号
├─ PartnerProxy_Main() // 合作伙伴组：20Hz — 第三方供应商信号
└─ mradar_dev_send_main_function() // 传感器组：20Hz — 雷达数据上报SOC
```

每个代理组有固定的文件模式：

- {Group}Proxy.c — 实现 {Group}Proxy_Main() 20Hz 主函数
- {Group}Proxy.h — 对外接口
- {Group}Proxy_Types.h — 信号收发数据结构定义

与 p301 的关键差异：p301 没有独立的 proxy group 目录结构，其传感器代理逻辑集中在 SOC 侧的 proxy_soc.cpp + MCU 侧的 xwire 文件中；e001_10 则使用标准化的 proxy group 模式。

七、SOC→MCU 反向数据流: IMU 传感器数据下发

这是 SOC 侧感知结果下发到 MCU 的典型路径，体现为 DCMS Topic 回调模式：

【SOC侧感知进程】

```
[
└─ dcms_adp_send_msg(DCMS_TOPIC_IMU_SENSOR, imu_data, sizeof(Imu), ...)
└─ // 平台仓 [dsar-plat-bf: dcms_mcu_api.h]
└─ // SOC通过DCMS将IMU数据(加速度+角速度)发送给MCU
]
▼
[X-Wire IPC ===== MCU侧]
[
▼
[vehicle_sensor_comm_init()] // vehicle_sensor.c:121-130
└─ dcms_mcu_topic_setup_rcv_msg_callback(
└─ DCMS_TOPIC_IMU_SENSOR,
└─ vehicle_sensor_imu_rcv_cb) // 平台仓: 注册IMU消息回调
└─ ctrl_imu_data_cb_register( // 平台仓 [dsar-bsw/ctrl]:
└─ vcu_perception_imu_sensor_rcv_msg_cb1) // 注册备用IMU回调

vehicle_sensor_imu_rcv_cb(msg) // DCMS回调: 收到SOC发来的IMU数据
└─ memcpy(&vehicle_sensor.sensor_get.imu, msg->data, sizeof(Imu))
└─ vehicle_sensor.sensor_get.imu_sensor_time = msg->sensor_time / 1e6
└─ // IMU数据存入全局 sensor_dev_t 结构体, 供其他模块读取

vcu_perception_imu_sensor_rcv_msg_cb1() // 备用回调: 带时间戳校验
└─ sensortime_invalid() 检查时间戳有效性

ComIPduCallout_TICE_General_Status_15_oCAN01_xxx_Rx() 是 AUTOSAR COM 层的 IPdu 回调函数 (定义在 Com_Cbk.h), 当特定 CAN 报文(如 ADASIS v3 协议的地图数据)到达时由 COM 栈自动触发。
```

八、MCU → SOC 数据上报: 雷达原始数据流

文件: mradar_obtain.c + mradar_info_send.c

```
// === 步骤1: CAN信号读取 (mradar_obtain.c) ===
mradar_dev_send_main_function(tick) // 20Hz
└─ zyt_mradar_com(Get_Com_Map_MRR()) // 前雷达数据
└─ MRR_Com_Get(i, N, MRR.xxx) // 宏展开为:
└─ Com_ReceiveSignal( // 平台仓 [dsar-bsw]: AUTOSAR COM读取
└─ MRR_Com_Map[obj_idx][sig_idx], &var)
└─ 物理值转换: raw * FACTOR + OFFSET // 适配仓: 车型标定参数
└─ 存入 g_st_mradar_msg.mradar_get.objects[i]
└─ 同样处理 FLR/FRR/RLR/RRR (前后左右4个角雷达)

// === 步骤2: DCMS上报SOC (mradar_info_send.c) ===
zyt_mradar_comm_send() // 20Hz
└─ zyt_radar_proxy_sensortime_update_period()
└─ sync_signal_aligned_time_us_by_freq() // 平台仓: 时间对齐到20Hz
└─ acu_dev_read(g_pstMRADARAcuDev, // 平台仓 [dsar-bsw]: 从设备抽象层
└─ ACU_MRADAR_GET_OBJ, ...) // 读取所有雷达目标
[
└─ zyt_radar_raw_dcms_send(
└─ DCMS_TOPIC_MRR_SIGNAL, data, len)
└─ zyt_radar_raw_data_dcms_send()
└─ dcms_mcu_topic_send_msg() // 平台仓: DCMS发送给SOC
└─ // SOC侧的 radar_proxy_main_function() 接收此数据
```

九、设备抽象层(ACU_DEV): MCU 的内部共享内存机制

acu_dev 是 MCU 各模块之间的 共享数据抽象层, 定义在平台仓 dsar-bsw 中:

```
// 注册设备 (如 vehicle_lat.c:614)
lat_dev_register(&vehicle_lat)
→ 将 vehicle_lat 指针注册到全局设备表

// 跨模块写入 (如 MCU shell 调试中: vehicle_lat.c:724-740)
acu_dev_write(&vehicle_lat.acu_dev,
ACU_LAT_SET_ANG_CTRL_DUTY, &ang_ctrl_duty, 4)
→ 其他模块可通过该信号ID写入横向控制指令

// 跨模块读取 (如 mradar_info_send.c:130-134)
acu_dev_read(g_pstMRADARAcuDev,
ACU_MRADAR_GET_OBJ, &objects, sizeof(objects))
→ 从 mradar_dev_t 全局实例中读取雷达目标数据
```

关键平台仓函数 (来自 dsar-bsw 子模块):

函数	作用
lat_dev_register() / lon_dev_register() / sensor_dev_register() / app_dev_register() / mradar_dev_register()	注册各类型设备到全局表
acu_dev_write() / acu_dev_read()	按信号ID跨模块读写设备数据
acu_dev_get(ACU_DEVICE_ID(...))	按设备和ID获取设备指针
get_sys_mode() / get_sec_sys_mode()	获取当前系统模式(调试/正常)

十、时间同步与信号对齐

SOC 和 MCU 之间通过时间同步机制保证数据一致性:

```
// MCU侧 (如 mradar_info_send.c:104)
sync_signal_aligned_time_us_by_freq(&gs_time_align,
get_time_us_32bits(), // 平台仓 [dsar-bsw]: 获取32位微秒时间戳
RADAR_RAW_DATA_DCMS_FREQUENCY) // 对齐频率 (如20Hz=50000us)
```


→ 将当前时间对齐到最近的20Hz整数倍边界
→ gs_time_align.aligned_time_stamp 为对齐后的时间戳

```
// 每个DCMS消息携带 sensor_time, SOC侧可以根据时间戳进行传感器融合
dcms_mcu_topic_send_msg(topic_id, &raw_msg)
// raw_msg.sensor_time = aligned_time_stamp
```

SOC 侧的对应平台仓函数 (在 sig_route_soc.cpp 中也用到了类似的 sync_signal_aligned_time_us_by_freq).

十一、录制子系统(Recorder)

文件: proxy_comm/recorder_common/ + 平台仓 cdd/recorder.h

```
// vehicle_lat.c:602
recorder_data(FAW_P301_LAT_CTRL_50HZ,
              (uint8_t *)&acu_lat_record_data,
              sizeof(acu_lat_record_t))
→ 平台仓 [dsar-plat-bf: recorder.h]
→ 将横向控制所有信号(含raw/physical value)记录到内部存储
→ 用于离线回放和问题诊断

// 各模块按需录制: comm_sensorimu_record_t, comm_sensorodo_record_t 等
```

十二、完整通信函数分类总结

通信方向	适配仓函数(位置)	调用的平台仓函数(来源)	平台仓作用
CAN → MCU	vehicle_lat_get() (xwire/vehicle_lat.c)	Com_ReceiveSignal() (dsar-bsw)	AUTOSAR COM栈读取CAN信号原始值
MCU → CAN	SigIf_Set_xxx() (生成代码) → vehicle_lat_set()	Com_SendSignal() (dsar-bsw)	AUTOSAR COM栈发送CAN信号
MCU → SOC (信号)	SigIf_Set_xxx() (生成代码) → msg buffer	dcms_mcu_topic_send_msg() (dsar-plat-bf)	跨核DCMS发送CAN信号给SOC
SOC → MCU (信号)	SigIf_Get_xxx() (SOC生成代码)	sig_route_main_function() → dcms_adp_send_msg() (dsar-plat-bf)	SOC通过DCMS下发控制指令给MCU
SOC → MCU (传感器)	vehicle_sensor_imu_rcv_cb() (xwire/vehicle_sensor.c)	dcms_mcu_topic_setup_rcv_msg_callback() (dsar-plat-bf)	注册DCMS回调接收SOC发来的IMU数据
MCU → SOC (雷达)	zyt_mradar_comm_send() (proxy_sensor/mradar_info_send.c)	dcms_mcu_topic_send_msg() (dsar-plat-bf)	雷达原始目标数据上报SOC
模块间数据共享	acu_dev_read/write() 调用	acu_dev_read/write() (dsar-bsw)	MCU内部全局设备数据表
时间同步	各模块周期性调用	sync_signal_aligned_time_us_by_freq() (dsar-bsw)	将时间对齐到指定频率
设备注册	xxx_dev_register() (各 xwire/vehicle_*.c)	lat/lon/sensor/app_dev_register() (dsar-bsw)	将设备实例注册到全局设备管理器
数据录制	recorder_data() (各模块)	recorder_data() (dsar-plat-bf)	信号数据录制到内部存储
CAN Gateway	CAN_GATEWAY_REG_TX_ID_PARAMS() (adapter/cangateway_adapter.c)	CAN-Gateway路由注册 (dsar-bsw)	SOC → MCU方向CAN帧转发

十三、两种产品变体的对比

维度	fawhq_p301	fawhq_e001_10
MCU proxy模式	无独立proxy group, 传感器逻辑在SOC proxy_soc.cpp	6个标准化proxy group (Actuator/BCM/Chassis/HMI/Partner/Sensor)
xwire文件数	10个 (含vehicle_perception/sensor/app等)	10个 (相同结构)
SOC侧结构	proxy/ + SensorGroup/ + x_dom_can_rt_gen_com/uds/	proxy/ + SensorGroup/ + x_dom_can_rt_gen_com/uds/ + x_dom_someip_rt_gen_com/uds/
OEM feature	无独立oem_product_feature目录	有 oem_product_feature/ (dump_adapter + someip_service)
DBC	3个DBC文件 (ADCM + ADAS + 通用)	—
信号方式	CAN FD + SOME/IP	CAN + SOME/IP

两种变体共享相同的 xwire/vehicle_*.c 文件结构, 差异主要在 proxy 组模式和生成代码的 JSON 配置上。